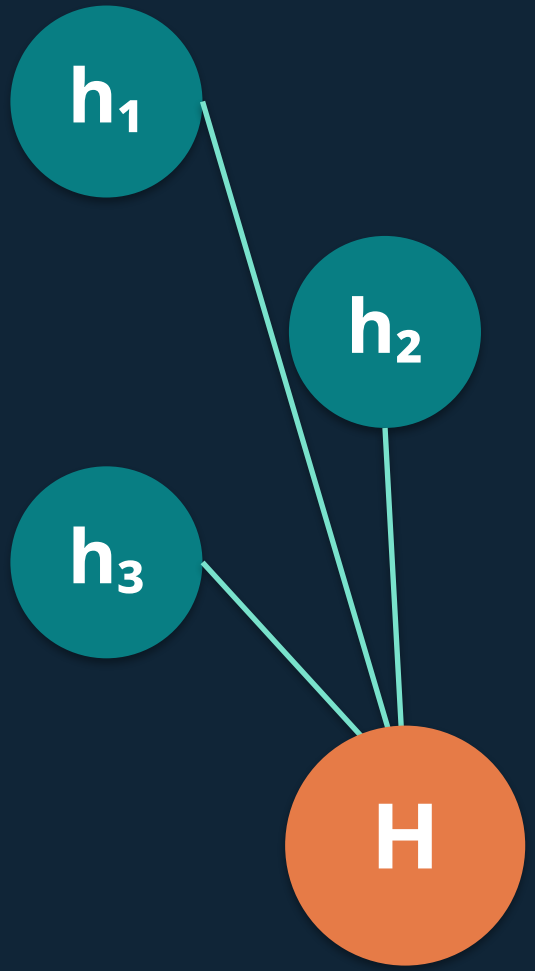
ENSEMBLE LEARNING

Từ cây quyết định đến mô hình tổ hợp

Đề 5 • Bagging và bỏ phiếu đa số

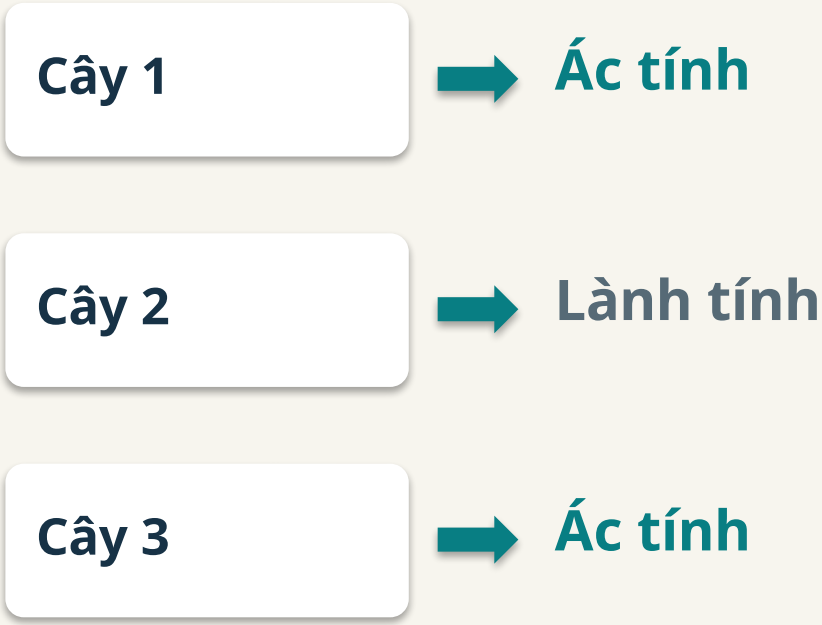
Nhóm: [Điền tên thành viên] | Lớp: [Điền lớp]



Nhiều mô hình chỉ hữu ích khi có sự đa dạng

Một mô hình có thể sai.
Một tập mô hình có thể bù lỗi cho nhau.

Điều kiện: mô hình cơ sở đủ tốt
và lỗi không hoàn toàn giống nhau.



Lộ trình: Nguyên lý → Ví dụ từng bước → Mã nguồn → Thực nghiệm

Bốn cách kết hợp thường gặp

BAGGING

Song song

Nhiều mẫu bootstrap
→ nhiều mô hình
→ tổng hợp dự đoán

Ví dụ: Bagged Trees

BOOSTING

Tuần tự

Mô hình sau tập trung
vào sai sót / phần dư
của mô hình trước

Ví dụ: AdaBoost, GBM

VOTING

Quy tắc kết hợp

Kết hợp nhãn dự đoán
hoặc xác suất lớp
của nhiều mô hình

Hard / Soft Voting

STACKING

Học cách kết hợp

Dự đoán ngoài fold
→ tạo đặc trưng mới
→ mô hình tầng hai

Meta-model

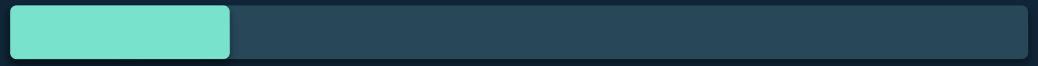
Đa dạng giúp giảm phương sai

$\text{Var(trung bình)} = \sigma^2 [\ \rho + (1 - \rho) / B\ ]$

B: số mô hình
σ^2: phương sai của mỗi mô hình
ρ: tương quan giữa hai mô hình

Thêm mô hình giảm phần (1 − ρ)/B.
Phần ρ vẫn tạo ra giới hạn:
các mô hình quá giống nhau ít có lợi.

B = 51, ρ = 0,2 → 0,216 σ^2



B = 51, ρ = 0,8 → 0,804 σ^2



Giả thiết: cùng phương sai và tương quan cặp chung; áp dụng cho trung bình dự đoán số.

Hard Voting và Soft Voting có thể cho kết quả khác

Quan sát x	Mô hình 1	Mô hình 2	Mô hình 3
P(ác tính | x)	0,51	0,51	0,01
Nhãn dự đoán	Ác tính	Ác tính	Lành tính

HARD → ÁC TÍNH

Đếm nhãn: 2 phiếu ác tính / 3 phiếu.

SOFT → LÀNH TÍNH

Trung bình P(ác tính) = 1,03 / 3 ≈ 0,343.

Bagging: lấy mẫu lại, học độc lập, rồi tổng hợp

01

Tập train

N quan sát
Giữ riêng test

02

Bootstrap

B mẫu; mỗi mẫu
N lượt có hoàn lại

03

Huấn luyện

Một cây / mẫu
Độc lập với nhau

04

Tổng hợp

Phiếu từng cây
→ lớp đa số

Thực nghiệm: N = 426 mẫu train • B = 51 cây • max_depth = 3

Ví dụ bootstrap: 5 quan sát, 5 lượt lấy mẫu

Tập gốc có chỉ số [0, 1, 2, 3, 4]. Mỗi dòng là một mẫu huấn luyện mới.

Cây	Chỉ số bootstrap	Không có mặt
h_1	[0, 0, 2, 4, 1]	3
h_2	[1, 3, 3, 2, 0]	4
h_3	[4, 2, 4, 1, 3]	0

CÓ LẶP

Cùng N lượt, nhưng không đủ N mẫu khác nhau.

$P(\text{một mẫu không được chọn}) = (1 - 1/N)^N \rightarrow e^{-1} \approx 36{,}8\%$ khi N lớn

Một cây học phép chia như thế nào?

Dữ liệu minh họa	0	1	2	3	4
Đặc trưng x	1	2	3	4	5
Nhãn	Lành	Lành	Ác	Ác	Ác

Bootstrap h_1: [0, 0, 2, 4, 1]
→ 3 lành tính, 2 ác tính

Gini trước chia
$= 1 - (3/5)^2 - (2/5)^2 = 0{,}48$

Chọn ngưỡng x ≤ 2,5

Trái: 3 lành → Gini = 0
Phải: 2 ác → Gini = 0
Gini sau chia = 0 | Giảm = 0,48

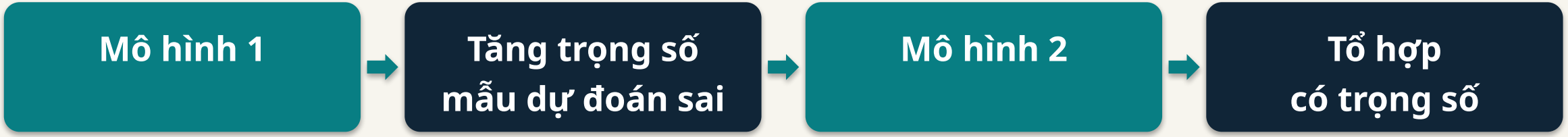
Từ phiếu riêng lẻ đến quyết định của tổ hợp

$$H(x) = \text{argmax}_c \sum_\beta I[h_\beta(x) = c]$$



Mã hóa: ác tính = 0, lành tính = 1. 51 cây tránh hòa phiếu ở bài toán hai lớp.

Boosting: mô hình sau tập trung vào sai sót trước



Ví dụ AdaBoost nhị phân

$\varepsilon = 0{,}20 \rightarrow \alpha = \frac{1}{2}\ln(0{,}8/0{,}2) \approx 0{,}693$

Mẫu sai: nhân $\exp(+\alpha) \approx 2$

Mẫu đúng: nhân $\exp(-\alpha) \approx 0{,}5$

Sau đó chuẩn hóa tổng trọng số về 1.

20% → 50%

Khối lượng trọng số nhóm sai:

$0{,}2 \times 2 / (0{,}2 \times 2 + 0{,}8 \times 0{,}5) = 0{,}5$.

Stacking học một mô hình để kết hợp các mô hình

TẦNG CƠ SỞ

Cây / SVM / Logistic...
Huấn luyện theo K fold.
Mỗi mẫu chỉ lấy dự đoán
từ mô hình chưa thấy nó.

ĐẶC TRƯNG OOF

Một cột / mô hình cơ sở.
Các dự đoán ngoài fold
tạo đầu vào cho
mô hình tầng hai.

META-MODEL

Học cách phối hợp
các mô hình cơ sở.
Test chỉ dùng để
đánh giá cuối cùng.

Tránh rò rỉ: không huấn luyện tầng hai bằng dự đoán in-sample của tầng cơ sở.

Bài toán: phân loại mẫu khối u vú

569
quan sát

30
đặc trưng số

212
ác tính (nhãn 0)

357
lành tính (nhãn 1)

ĐẦU VÀO

Đặc trưng nhân tế bào: radius, texture, perimeter, area...

ĐẦU RA

Dự đoán ác tính hoặc lành tính cho một quan sát mới.

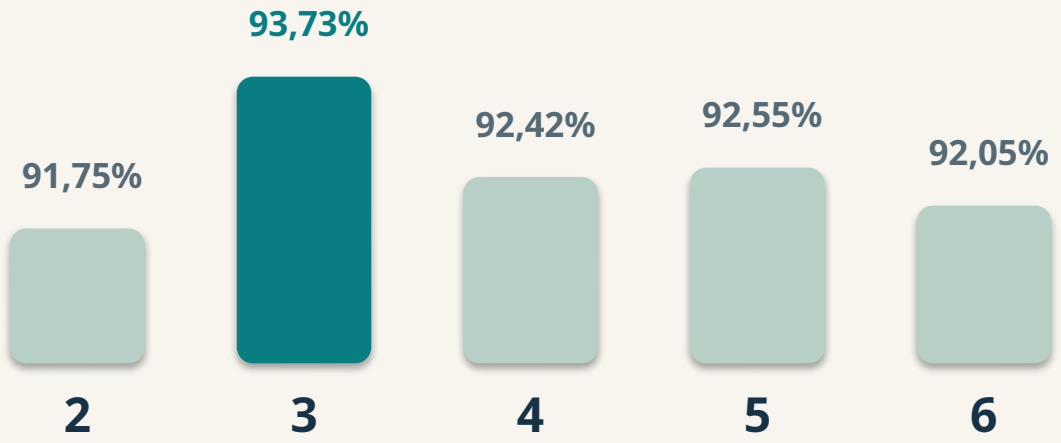
Mục đích học thuật; chưa xác thực lâm sàng. Dữ liệu có sẵn trong scikit-learn.

Thiết kế thí nghiệm giữ test ngoài quá trình chọn độ sâu

TRAIN 75% • 426 mẫu (159 ác / 267 lành)

TEST • 143 mẫu

CV phân tầng 5 fold trên train



Trục độ sâu 2–6; cột hiển thị phần vượt 90% CV.

CHỌN max_depth = 3

Balanced accuracy CV = 93,73%.
Chọn bằng cây đơn, dùng chung cho cả ba mô hình.
Không tối ưu riêng từng ensemble.

Nguồn: outputs/results.json • seed = 42

Tự viết hai phần cốt lõi: bootstrap và voting

01 LẤY MẪU & HUẤN LUYỆN

```
indices = rng.integers(
    0, len(X), size=len(X)
)
tree = DecisionTreeClassifier(
    max_depth=self.max_depth,
    random_state=...
)
tree.fit(X[indices], encoded[indices])
self.trees_.append(tree)
```

02 ĐẾM PHIẾU & CHỌN LỚP

```
predictions = np.asarray([
    tree.predict(X)
    for tree in self.trees_
])
# Với mỗi lớp, đếm số cây chọn lớp đó
np.sum(predictions == label, axis=0)

# Dự đoán cuối cùng
self.classes_[votes.argmax(axis=1)]
```

Cây cơ sở dùng thư viện; quy trình Bagging và hard voting do mã minh họa triển khai.

Bagging tự viết và Random Forest khác nhau ở đâu?

Thành phần	Manual Bagging	Random Forest (sklearn)
Lấy mẫu train	Bootstrap	Bootstrap
Đặc trưng khi chia	Xét toàn bộ 30 đặc trưng	Tập con ngẫu nhiên (√30 → 5)
Kết hợp cây	Đa số nhãn dự đoán	Trung bình xác suất lớp
Cấu hình thí nghiệm	51 cây, max_depth = 3	51 cây, max_depth = 3

Ngẫu nhiên hóa đặc trưng giúp giảm tương quan; không bảo đảm mọi cây sẽ có lỗi khác nhau.

[2], [3] • RandomForestClassifier mặc định max_features='sqrt'

Bagging cải thiện nhẹ trên lần chia test này

Mô hình	Accuracy	Precision*	Recall*	F1*
Cây quyết định đơn	94,41%	94,12%	90,57%	92,31%
Bagging tự viết	**95,10%**	**94,23%**	**92,45%**	**93,33%**
Random Forest	95,10%	94,23%	92,45%	93,33%

* Lớp dương: ác tính = 0. Precision, Recall và F1 đều tính cho lớp này.

Accuracy tăng 0,70 điểm phần trăm; không suy ra ưu thế tổng quát từ một lần chia.

Nguồn: outputs/results.json • test n = 143 • số liệu đã làm tròn

Nhìn vào ca bỏ sót, không chỉ accuracy

CÂY ĐƠN

	Dự đoán ác	Dự đoán lành
Thật ác	48	5
Thật lành	3	87

BAGGING

	Dự đoán ác	Dự đoán lành
Thật ác	49	4
Thật lành	3	87

Bỏ sót ác tính: 5/53 → 4/53 | Recall: 90,57% → 92,45%

Bagging vẫn bỏ sót 4 ca; chưa đủ cơ sở sử dụng cho quyết định lâm sàng.

Hàng = nhãn thật • Cột = dự đoán • Lớp dương = ác tính (0)

Theo dấu một dự đoán thật trong chương trình

Quan sát đầu tiên của test • Nhãn thật: lành tính

10 chỉ số train đầu trong bootstrap cây 1

```
[38, 329, 278, 186, 184,
 365, 36, 297, 85, 40]
```

Cả 5 cây đầu đều bỏ phiếu lành tính.

TỔNG 51 CÂY

0 : 51

Ác tính : Lành tính

Dự đoán tổ hợp: lành tính → Đúng với nhãn thật của quan sát này.

Kết quả có ích, nhưng phạm vi kết luận còn hẹp

ĐÃ KIỂM CHỨNG

Mã chạy được.
Bootstrap và hard voting được triển khai rõ ràng.
Có test độc lập.
Có số đo và phân tích lỗi.

GIỚI HẠN

Một seed, một test split.
Chọn depth bằng cây đơn.
Chưa đánh giá OOB.
Chưa kiểm định ngoài mẫu trên quần thể độc lập.

PHÁT TRIỂN

Lặp lại nhiều seed.
Báo cáo độ biến thiên.
Tuning riêng bằng CV.
So sánh AdaBoost, GBM.
Đánh giá đánh đổi lỗi.

Demo: chạy một lệnh, xem ba loại đầu ra

```
cd /workspace/hocmaygk
python3 de_5_ensemble_learning.py
```

metrics.csv

Bảng accuracy, precision, recall, F1 của ba mô hình.

results.json

Cấu hình, điểm CV, trace bootstrap và voting.

Ma trận nhầm lẫn

confusion_matrices.png
Đọc nhãn và thứ tự trục.

Cài dependencies theo README trước khi chạy. Không cần tải dữ liệu hoặc khóa API.

Ba điều cần nhớ

1

Đa dạng trước, tổng hợp sau

Nhiều mô hình giống nhau không tự tạo ra một tổ hợp tốt.

2

Bagging = Bootstrap + Aggregating

Mẫu có hoàn lại tạo các cây khác nhau; voting kết hợp dự đoán.

3

Đọc lỗi cụ thể, giữ kết luận có giới hạn

Bagging đạt 95,10% accuracy và bỏ sót 4/53 ca ác tính trên test này.

Tài liệu tham khảo

[1] Breiman (1996). Bagging Predictors. Machine Learning 24, 123–140.
https://doi.org/10.1007/BF00058655

[2] Breiman (2001). Random Forests. Machine Learning 45, 5–32.
https://doi.org/10.1023/A:1010933404324

[3] scikit-learn — Ensemble methods (Voting, Bagging, RF, Boosting, Stacking).
https://scikit-learn.org/stable/modules/ensemble.html

[4] scikit-learn — Breast Cancer Wisconsin Diagnostic dataset.
https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_breast_cancer.html

[5] scikit-learn — Decision Trees (Gini impurity, splitting criteria).
https://scikit-learn.org/stable/modules/tree.html

[6] Freund & Schapire (1997). A Decision-Theoretic Generalization...
https://doi.org/10.1006/jcss.1997.1504

[7] Wolpert (1992). Stacked Generalization. Neural Networks 5(2), 241–259.
https://doi.org/10.1016/S0893-6080(05)80023-1

Số liệu thực nghiệm: outputs/results.json • Seed 42 • NumPy 2.3.5 / scikit-learn 1.8.0

Câu hỏi thường gặp khi bảo vệ

Vì sao 51 cây?	Minh họa tổ hợp đủ lớn, tránh hòa ở hai lớp; chưa tối ưu số cây.
Vì sao không chuẩn hóa?	Các cây dùng phép chia theo ngưỡng, không dựa vào khoảng cách Euclid.
Test có bị dùng để tuning?	Không. Độ sâu chọn bằng CV trên train; test chỉ đánh giá cuối.
Bagging có luôn tốt hơn?	Không. Lỗi tương quan cao và mô hình cơ sở yếu có thể làm lợi ích thấp.
Vì sao nhấn mạnh recall?	Recall ác tính phản ánh tỷ lệ phát hiện và số ca ác tính bị bỏ sót.

Cách tính chỉ số với ác tính là lớp dương

Chỉ số	Công thức	Thay số Bagging
Accuracy	(TP + TN) / N	(49 + 87) / 143 = 95,10%
Precision ác tính	TP / (TP + FP)	49 / (49 + 3) = 94,23%
Recall ác tính	TP / (TP + FN)	49 / (49 + 4) = 92,45%
F1 ác tính	2TP / (2TP + FP + FN)	98 / (98 + 3 + 4) = 93,33%

Balanced accuracy dùng trong CV = (Recall ác tính + Recall lành tính) / 2.

Số đếm của Bagging: TP = 49, FN = 4, FP = 3, TN = 87